

Automatização e Unificação de Verificação Formal de Hardware

Uma Abordagem Baseada em AST para Validação VHDL-C.

01

Contextualização



O cenário atual da verificação formal e a lacuna entre
Hardware e Software

Contextualização



Cenário

A validação de circuitos críticos exige prova matemática (não apenas testes).



A Lacuna Técnica

Dificuldade em traduzir fielmente Hardware (VHDL) para Software (C).



O Problema de Mercado

Ferramentas proprietárias são caras; *open-source* é fragmentado.



Nossa Abordagem

Um pipeline que integra design industrial (Quartus) com verificação de software (ESBMC).

02

O Desafio Técnico

Inconsistência de largura de bits (Bitwidth Consistency) e
falsos positivos

O Desafio Técnico



Inconsistência de Largura de Bits

O Hardware (VHDL) usa vetores finitos (ex: 8 bits); o Software (C) usa inteiros de arquitetura (32/64 bits).



O Erro Gerado

Falsos positivos de *overflow*. O software alerta erro onde o hardware apenas realiza *wrap-around* (volta ao zero).



Impacto

A discrepância matemática inviabiliza a confiança na verificação automática.

Objetivos

Mitigação de erros, automação do fluxo e unificação de front-end

Objetivos



Mitigação

Eliminar erros semânticos na tradução VHDL-C.



Unificação

Criar um *Front-end* Unificado que garanta consistência de tipos.



Automação

Orquestrar todo o fluxo (tradução, síntese e verificação) sem intervenção manual.



Contraprova

Usar verificação de hardware (Symbi Yosys) em paralelo para validar a "verdade física".

04

Engenharia do Projeto

Infraestrutura WSL, OSS CAD Suite e ferramentas utilizadas.

Engenharia do Projeto



WSL 2

Uso mandatório do Ubuntu no Windows para rodar ferramentas nativas de Linux.

Intel Quartus Prime

Pré-validação (*Sanity Check*) e garantia de síntese industrial.

OSS CAD Suite

GHDL (Análise) + Yosys (Síntese e Extração de Dados).

ESBMC

Verificador de modelos de software (*Bounded Model Checking*).

05

O Pipeline Automatizado

Da síntese RTL à verificação dual.

O Pipeline Automatizado



Passo 1

Parser de tags e Síntese
RTL (Geração do
Hardware).



Passo 4

Geração do Código C
para o verificador de
software.



Passo 2

Verificação Formal de
Hardware (Symbi Yosys)
→ Define o "Ground
Truth".



Passo 3

Extração da AST (Árvore
de Sintaxe) em JSON via
Yosys.

A Solução (AST)

Geração dinâmica de máscaras e correção matemática via JSON.

A Solução (AST)



Análise Dinâmica

O script lê o JSON para descobrir a largura real dos fios (ex: data = 8 bits).



Efeito

O Software passa a imitar o comportamento de *wrap-around* do Hardware.



Correção Matemática

Injeção automática de máscaras (ex: & 0xFF) no código C.



Resultado

Eliminação total dos falsos positivos de incompatibilidade de tipos.

07

Resultados e Conclusão

Validação cruzada, convergência de verificadores e impacto final.

Resultados e Conclusão



Validação Cruzada

Convergência total entre Hardware (SBY) e Software (ESBMC).



Robustez

O sistema validou *constructs* complexos (acumuladores) sem erros falsos.



Impacto

Criação de um fluxo de verificação robusto, gratuito e matematicamente confiável.